

Build Task — Reusable Chain Module — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python).

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building three small files that work together: one that describes the prompt, one that builds the chain, and one that compares the chain to Project 1's raw version.

The chain's job: take the same input Project 1's raw feature takes, and produce the same kind of structured output — just built with `|` instead of raw API calls.

The comparison script's job: run both versions on the same inputs and prove, not just claim, that they agree.

Things to look up and use: - `ChatPromptTemplate.from_template(...)` — build your prompt in `prompts.py` - `.with_structured_output(YourModel)` — attach the same Pydantic shape Project 1 already uses. - `|` — chain the prompt and the structured model together. - Your `Doc01.load_config()` — read the model name and any settings from there, not hardcoded. - Project 1's raw extraction function — import it directly into `compare_with_raw_sdk.py`, don't rewrite it.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

You're building two separate, reusable things that get used together: a `prompts.py` module holding the prompt template(s), and a `chain.py` module that builds and returns the finished `Runnable`. Keep them separate — later documents will import `chain.py`'s function without caring how the prompt itself is written.

The key requirement to hold onto: the model name and temperature must come from your Doc01 config, never hardcoded. This means `build_extraction_chain()` should call `load_config()` (or accept a config object as a parameter) rather than writing `ChatOpenAI(model="gpt-4o-mini")` directly inside it.

Here's what to actually go look at: - `prompts.py` should hold a function like `get_extraction_prompt() -> ChatPromptTemplate`, not a bare module-level variable — this makes it easier to test and to swap later without touching `chain.py`. - `chain.py`'s `build_extraction_chain() -> Runnable` should call `load_config()` from Doc01 (or accept a `Config` object as a parameter — decide which and be consistent). - **Reusing Project 1's Pydantic model**, not redefining it, is what actually makes the comparison meaningful. - `compare_with_raw_sdk.py` should print a clear pass/fail per test input, and print a clear summary line if anything mismatched.

Sketch, in plain English, the exact function signature of `build_extraction_chain()`, then compare against Hint 2.

Difference between Basic and Intermediate: Basic names the 3 files and the tools each needs. Intermediate adds the real function signatures and the "config from `load_config()`, never hardcoded" constraint — this is the depth the Solution is written at.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
prompts.py:
    function get_extraction_prompt():
        return a ChatPromptTemplate with the same wording as Project 1's raw prompt

chain.py:
    function build_extraction_chain():
        load config (for model name, temperature)
        get the prompt from prompts.py
        build a ChatOpenAI with the config's settings
        attach with_structured_output using Project 1's Pydantic model
        return prompt | structured_model

compare_with_raw_sdk.py:
    import Project 1's raw function
    import build_extraction_chain

    make a list of test inputs
    chain = build_extraction_chain()

    for each input:
        raw_result = call the raw function
        lcel_result = call chain.invoke
        print both, print whether they match
```

The trickiest part — wiring config into the chain instead of hardcoding it:

```
from config import load_config
from langchain_openai import ChatOpenAI

def build_extraction_chain():
    config = load_config()
    model = ChatOpenAI(model=config.model_name, temperature=config.temperature)
    # ... attach prompt and structured output here
```

Add a `model_name` and `temperature` field to your Doc01 `Config` if it doesn't already have them.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
from config import load_config, Config
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.runnables import Runnable

def build_extraction_chain(config: Config | None = None) -> Runnable:
    if config is None:
        config = load_config()
    model = ChatOpenAI(model=config.model_name, temperature=config.temperature)
    # ... attach the prompt template and .with_structured_output(...) here
```

Notice the `config: Config | None = None` parameter — this lets tests pass in a fake config without touching real environment variables, while normal callers can just call `build_extraction_chain()` with no arguments.

Try finishing the rest yourself, including `compare_with_raw_sdk.py`'s result-collection shape:

```
from dataclasses import dataclass

@dataclass
class ComparisonResult:
    input_text: str
    raw_result: "ExtractedData"
    lcel_result: "ExtractedData"
    matched: bool
```

Write `run_comparison(test_inputs) -> list[ComparisonResult]` as a separate function from `main()` — this is what lets the exact same logic be reused elsewhere without duplicating the comparison loop. `main()` should just call `run_comparison()`, then report the results. Try writing both before checking the Solution.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Two working versions below, Basic and Intermediate. Both are correct — read both, and think about which one you'd actually pick and why.

Basic Version

Approach 1 — one flat `chain.py`

```
# prompts.py
from langchain_core.prompts import ChatPromptTemplate

def get_extraction_prompt():
    return ChatPromptTemplate.from_template(
        "Extract structured data from the following text: {text}"
    )
```

```
# chain.py
from config import load_config
from langchain_openai import ChatOpenAI
from prompts import get_extraction_prompt
from project_1 import ExtractedData  # reuse Project 1's Pydantic model

def build_extraction_chain():
    config = load_config()
    prompt = get_extraction_prompt()
    model = ChatOpenAI(model=config.model_name, temperature=config.temperature)
    structured_model = model.with_structured_output(ExtractedData)
    return prompt | structured_model
```

```
# compare_with_raw_sdk.py
from chain import build_extraction_chain
from project_1 import extract_raw

test_inputs = ["input one", "input two", "input three"]
chain = build_extraction_chain()

for text in test_inputs:
    raw_result = extract_raw(text)
```

```
lcel_result = chain.invoke({"text": text})  
print(text, "-> match:", raw_result == lcel_result)
```

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — config passed as an optional parameter

chain.py

```
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI

from config import Config, load_config
from prompts import get_extraction_prompt
from project_1 import ExtractedData

def build_extraction_chain(config: Config | None = None) -> Runnable:
    if config is None:
        config = load_config()

    prompt = get_extraction_prompt()
    model = ChatOpenAI(model=config.model_name, temperature=config.temperature)
    structured_model = model.with_structured_output(ExtractedData)

    return prompt | structured_model
```

compare_with_raw_sdk.py

```
from dataclasses import dataclass

from chain import build_extraction_chain
from project_1 import extract_raw, ExtractedData

@dataclass
class ComparisonResult:
    input_text: str
    raw_result: ExtractedData
    lcel_result: ExtractedData
    matched: bool

def run_comparison(test_inputs: list[str]) -> list[ComparisonResult]:
```

```

chain = build_extraction_chain()
results = []
for text in test_inputs:
    raw_result = extract_raw(text)
    lcel_result = chain.invoke({"text": text})
    results.append(ComparisonResult(text, raw_result, lcel_result, raw_result))
return results

def main() -> None:
    test_inputs = ["input one", "input two", "input three"]
    results = run_comparison(test_inputs)

    matched = sum(1 for r in results if r.matched)
    print(f"{matched}/{len(results)} matched")

    for r in results:
        if not r.matched:
            print(f"MISMATCH on {r.input_text!r}: raw={r.raw_result} lcel={r.lcel_result}")

if __name__ == "__main__":
    main()

```

Difference from Basic: Basic gets the comparison working with plain per-line prints. Intermediate separates `run_comparison()` (the logic) from `main()` (the reporting), and collects results into a typed `ComparisonResult` list instead of printing inline — this is what makes the script reusable as an actual test later, not just a one-off script.

Which one should you actually use?

Basic's flat version is fine for getting the exercise working. Intermediate's optional-config-parameter pattern is the standard real-world default — convenient to call, still testable — and it's what this Build Task actually requires: an LCEL chain wired to config, and a script that proves it matches the raw-SDK version. Turning `compare_with_raw_sdk.py` into a real `pytest` test (parametrized over the same test inputs, `assert` ing equality) is worth doing once this module is something CI actually needs to catch drift on automatically — not required for the Build Task itself.